

Assignment 1: Exploratory Data Analysis on Fraud Detection Transactions Dataset

Dataset: `fraud_detection_transaction_dataset`

Total Marks: 60

Introduction

You are a **Data Analyst at SafeBank**, tasked with analyzing recent transaction records to detect patterns, identify anomalies, and prepare the data for fraud detection modeling. The dataset reflects **real-world banking practices** and contains transaction data with details such as transaction amount, device type, card information, account balance, location and historical fraud activity.

Your goal is to **explore, clean, and analyze the dataset** using Python's Pandas library and basic EDA techniques. This exercise will strengthen your practical skills in handling **messy and inconsistent datasets**, detecting outliers, and preparing data for downstream analysis.

Section 1: Introduction to Pandas & Basic Exploration

(5 Marks)

Task:

1. Import the dataset `fraud_detectiontransaction_dataset.csv`.
2. Display the first 10 rows (`head`) and last 10 rows (`tail`).
3. Get general information about the dataset using `.info()` and `.describe()`.
4. Identify **numeric** and **categorical** columns.

Questions:

- Which column(s) would you convert to datetime type for analysis? Why?
- How would you check the **total number of unique users** in the dataset?

Section 2: Selecting Data with loc and iloc (5 Marks)

Task:

1. Select the **Transaction_ID**, **User_ID**, and **Transaction_Amount** columns for the first 15 rows using `.iloc`.
2. Select all transactions where the `Location` is `New York` using `.loc`.
3. Select all rows where `Transaction_Amount > 5000` and `Risk_Score > 0.8`.

Questions:

- How can you select the last 5 columns for all rows without naming them individually?
 - What is the difference between `.loc` and `.iloc`? Give an example.
-

Section 3: GroupBy Operations (10Marks)

Task:

1. Calculate the **average transaction amount per Location**.
2. Find the **total number of fraudulent transactions (`Fraud_Label=1`) per Transaction_Type**.
3. Compute the **average Risk_Score per Device_Type and Card_Type combination**.

Questions:

- Which aggregation function would you use if you want **both mean and sum** of `Transaction_Amount` for each `Location`?
 - How can you sort the grouped data by descending order of average `Risk_Score`?
-

Section 4: Data Types (object, numeric, datetime) (10 Marks)

Task:

1. Identify all **object type** and **numeric type** columns.
2. Convert `Timestamp` to datetime type.
3. Create a new column `Hour_of_Day` extracted from `Timestamp`.

Questions:

- Why is it important to convert timestamp columns to datetime before analysis?
 - Which column types are suitable for outlier detection, and which are not?
-

Section 5: Outlier Detection & Solutions (10 Marks)

Task:

1. Detect outliers in `Transaction_Amount` using **boxplots** and **IQR method**.
2. Detect outliers in `Account_Balance`.

Questions:

- Why is it not always a good idea to remove all outliers?
 - How would you detect outliers in a categorical column like `Device_Type`?
-

Section 6: Missing Values Detection & Solutions (10 Marks)

Task:

1. Identify columns with missing values and calculate their **percentage**.
2. Fill missing **numeric values** (`Transaction_Amount`, `Account_Balance`) with **mean**.
3. Fill missing **categorical values** (`Device_Type`, `Location`, `Merchant_Category`) with **mode**.
4. For `Timestamp`, suggest a strategy to handle missing values for time-based analysis.

Tricky Questions:

- What happens if you fill missing values with mean in a skewed column?
 - How can you flag missing values for further analysis without filling them?
-

Section 7: Inconsistency Detection & Handling (10 Marks)

Info: Inconsistencies can appear as **duplicate IDs**, **invalid entries**, **unrealistic values**, or **contradictions** between related columns. Detecting and correcting these is essential before meaningful analysis.

Task:

1. **Duplicate Checks:**
 - Check for **duplicate Transaction_IDs** and duplicate rows; remove if necessary.
2. **Invalid or Inconsistent Categorical Values:**
 - Inspect `Transaction_Type`, `Device_Type`, `Location`, `Merchant_Category`, `Card_Type`, `Authentication_Method` for typos or invalid entries. Standardize values.
3. **Contradictory Data:**
 - Check for contradictions such as `Transaction_Amount > Account_Balance`.
 - Flag rows where `Previous_Fraudulent_Activity=0` but `Fraud_Label=1`.
4. **Unrealistic Values:**
 - Check for negative or zero values in `Transaction_Amount` or `Account_Balance`. Correct/remove if needed.
5. **Data Type Mismatch:**
 - Ensure numeric columns (`Transaction_Amount`, `Account_Balance`, `Risk_Score`) are numeric.
 - Ensure `Timestamp` is datetime and `Fraud_Label` is binary.

Questions:

- Why is it risky to assume all categorical values are valid without checking?
- How would you handle a `Transaction_Amount` that is extremely higher than all others—outlier or inconsistency?
- If a `User_ID` appears multiple times, how can you differentiate multiple transactions vs a data entry error?

Section 9: Get More Insights (Bonus Section)

Info: After cleaning and exploring the dataset, you can look for additional patterns or insights that help understand user behavior and fraud trends. This section encourages practical EDA and basic analysis to discover meaningful patterns.

Tasks:

1. User Behavior Analysis

- Find the **top 5 users** with the highest total `Transaction_Amount`.
- Find users who perform **most transactions during weekends** (`Is_Weekend=1`).

2. Risk and Fraud Patterns

- Count the number of fraudulent (`Fraud_Label=1`) transactions for each `Device_Type`.
- Count the number of fraudulent transactions for each `Location`.

3. Time-Based Analysis

- Count the number of transactions per **day** or **month** using `Timestamp`.
- Identify the **hour of day** when most high-value transactions occur (`Transaction_Amount > 5000`).

4. Derived Column Analysis

- Create a column `High_Risk_Transaction = 1` if `Risk_Score > 0.8` and `Transaction_Amount > 5000`, else 0.
- Calculate the **percentage of high-risk transactions** overall.

Questions:

- How can a few very high `Transaction_Amount` values affect the top 5 users analysis?
- If a user has many small transactions and one high transaction flagged as fraud, what does it tell you about their risk?
- Which simple patterns could help detect risky transactions in the future?

Submission Guidelines

- Use **Google Colab**..
- Use **separate sections** for each part of the assignment.
- Include **markdown cells** to describe your steps, assumptions, and insights.
- Ensure all code is **well-commented** explaining what each step does.
- All outputs should be **clear and visible**.